CodeQuiz

Assignment 6: jQuery

Team: Team Spirit

Members: Darkhan Saduakasov; Kassymkhan

Group: SE 2508

Website URL

https://arxqar-jpg.github.io/WEB-Technologies/index.html

Objective

Continue the existing team project with jQuery live search, autocomplete, search highlighting, scroll progress, animated counters, submit feedback, notifications, clipboard copying and image lazy loading.

Scope

The five pages use the selected Forest platform layout with local Sora/Figtree fonts. The same planner, search, forms, statistics and clipboard features remain. Quizzes contain nine practical code-reading questions. The earlier responsive cards, Bootstrap grids, carousel and FAQ are accessible through Resources & FAQ on Overview.

Build and deployment target

Prepared and tested on 4 November 2026. Deployment target: the existing team GitHub Pages site linked above. The team publishes this revision by committing and pushing the updated project files to the shared repository.

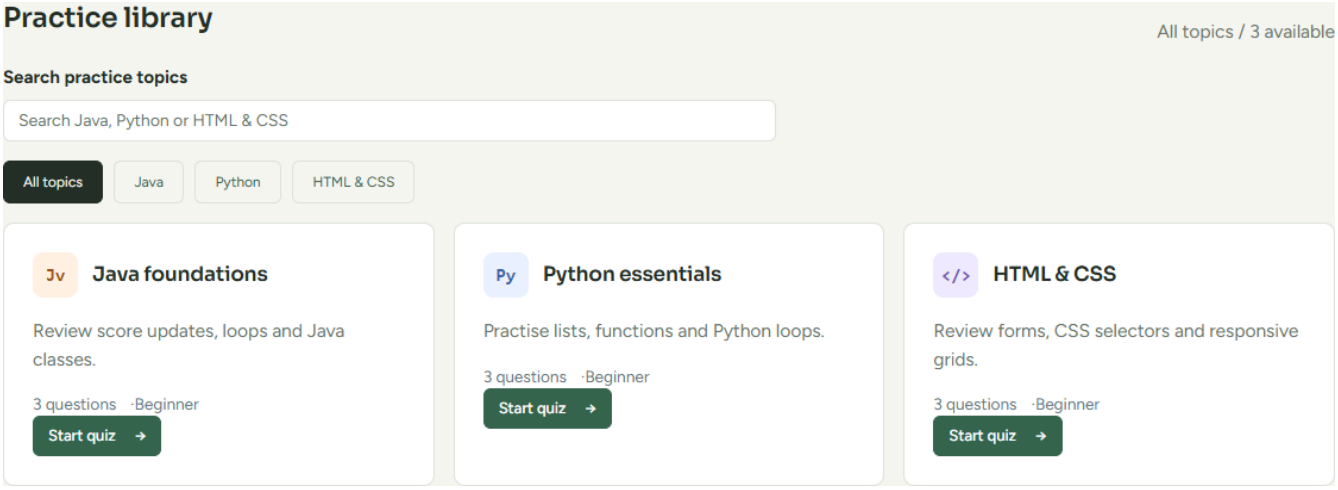
Task 0 / jQuery setup

1. Add the required jQuery 4.0.0 CDN script in the head of all five HTML pages. Keep a local copy of the same version as an offline fallback.

2. Use an external $(document).ready() block in js/script.js and print jQuery is ready! in the browser console.

3. Load the custom scripts after jQuery. Replace the old event shorthand calls with .on() for jQuery 4 compatibility. The browser tests confirmed the ready message, CDN HTTP 200 and offline initialization on every page.

Result in the updated webpage



Source-code screenshot (selected lines)

```
     study.html : 15
15   <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>
16   <script src="js/jquery-fallback.js"></script>
     js/script.js : 494
494  $(document).ready(function () {
495      console.log("jQuery is ready!");
496  });
     js/jquery-fallback.js : 2
2    if (!window.jQuery) {
3        document.write('<script src="vendor/jquery/jquery-4.0.0.min.js"><\/script>');
4    }
```

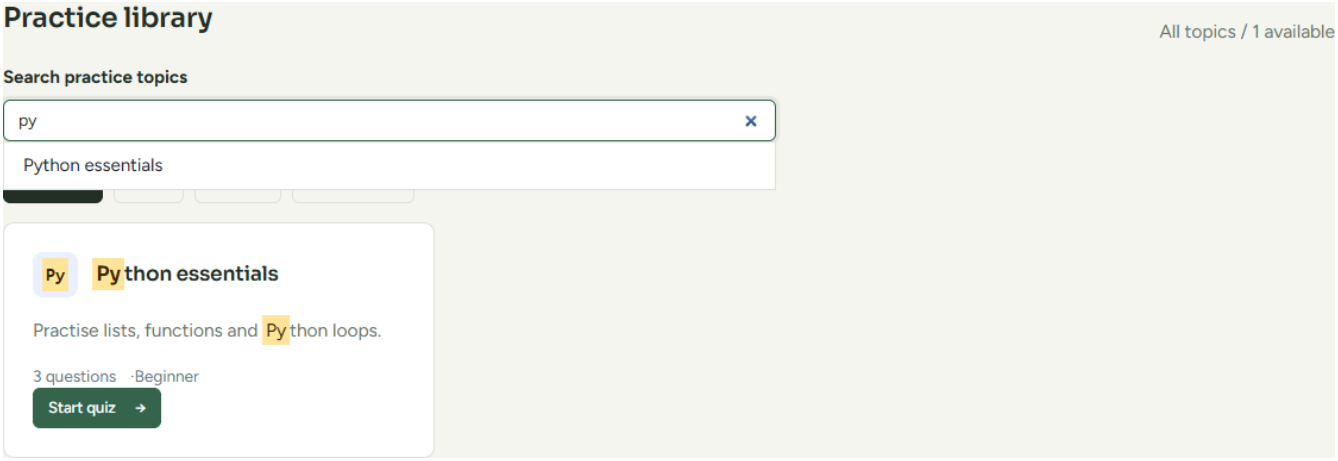
Tasks 1-2 / Live search and autocomplete

1. Task 1: type into Overview or Study Lab > Practice library > Search practice topics. Filter the existing topic objects by title/description while preserving the selected category.

2. Rebuild the matching cards immediately on input, update the result count and show a no-results message when necessary. Clearing the search restores the selected category.

3. Task 2: display matching topic titles in a dropdown. Selecting a suggestion completes the search. Arrow keys and Enter select suggestions; Escape closes the dropdown and preserves the query.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment5.js : 72
72  var query = $("#topic-search").val().trim().toLowerCase();
73  var visibleTopics = topics.filter(function (topic) {
74      return matchesCategory(topic) &&
75          (topic.title + " " + topic.description).toLowerCase().indexOf(query) !== -1;
76  });
77  var container = $("#practice-topics");
    js/assignment5.js : 111
111 function showSuggestions() {
112     var query = $("#topic-search").val().trim().toLowerCase();
113     closeSuggestions();
114     if (!query) return;
115     var list = $("#topic-suggestions");
116     for (var i = 0; i < topics.length; i++) {
117         var topic = topics[i];
118         if (matchesCategory(topic) && (topic.title + " " +
    topic.description).toLowerCase().indexOf(query) !== -1) {
119             var option = $("<li>", { role: "option", id: "topic-option-" + i, "aria-
    selected": "false" });
120             $("<button>", { type: "button", class: "list-group-item list-group-item-
    action", tabindex: -1 }).text(topic.title).appendTo(option);
121             list.append(option);
122         }
123     }
124     if (list.children().length) {
125         list.prop("hidden", false);
    js/assignment5.js : 130
130 $("#topic-search").on("input", function () {
131     renderTopics(currentCategory);
132     showSuggestions();
133 }).on("focus", showSuggestions).on("keydown", function (event) {
```

Task 3 / Search highlighting

1. After rendering results, find every literal case-insensitive match in the result-card text, including the title and description.

2. Escape regular-expression characters in the query. Walk text nodes and create mark elements with textContent; user input is not inserted as HTML.

3. Re-render cards for each search so previous highlights disappear and nested marks do not accumulate. The Python result contains two highlighted occurrences.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment6.js : 58
58  function highlightMatches(element, query) {
59      if (!query) return;
60      var expression = new RegExp(query.replace(/[.*+?^${}()|[\]\\]/g, "\\$&"), "gi");
61      $(element).contents().each(function () {
62          if (this.nodeType === 1) {
63              if (!$(this).is("script, style, mark")) highlightMatches(this, query);
64          } else if (this.nodeType === 3) {
65              var text = this.nodeValue;
66              var fragment = document.createDocumentFragment();
    js/assignment6.js : 70
70  while ((match = expression.exec(text)) !== null) {
71      fragment.appendChild(document.createTextNode(text.slice(start, match.index)));
72      var mark = document.createElement("mark");
73      mark.textContent = match[0];
74      fragment.appendChild(mark);
75      start = match.index + match[0].length;
76  }
77  if (start) {
78      fragment.appendChild(document.createTextNode(text.slice(start)));
79      this.parentNode.replaceChild(fragment, this);
```

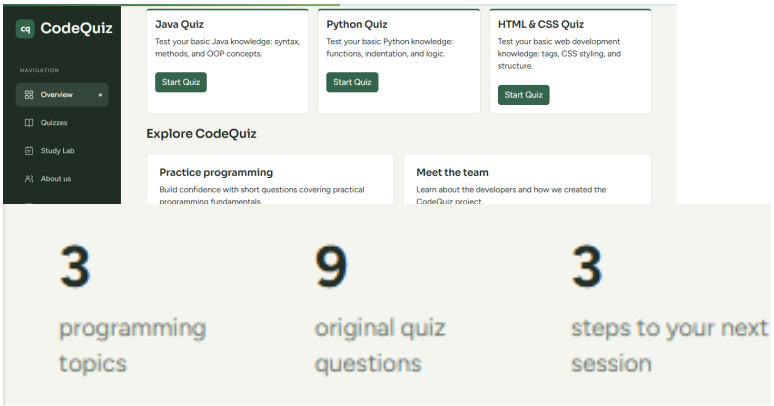
Tasks 4-5 / Scroll progress and number counters

1. Task 4: show a fixed gradient progress bar at the top of each page. Divide scrollTop by the available scroll distance and update the fill width and aria-valuenow on scroll/resize.

2. Task 5: animate the existing Study Lab totals from zero to 3 topics, 9 original quiz questions and 3 planner steps. These are actual project totals.

3. Start each counter once when it enters view and honor reduced-motion preferences. Tests checked intermediate values and exact final totals.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment6.js : 8
8   function initScrollProgress() {
9       function update() {
10          var distance = $(document).height() - $(window).height();
11          var percent = distance > 0 ? $(window).scrollTop() / distance * 100 : 0;
12          percent = Math.max(0, Math.min(100, percent));
13          $("#scroll-progress").css("width", percent + "%").attr("aria-valuenow",
    Math.round(percent));
14      }
15      $(window).on("scroll resize load", update);
16      $(document).on("slid.bs.carousel shown.bs.collapse", update);
17      update();
18  }
19

    js/assignment6.js : 27
27      $({ value: 0 }).animate({ value: target }, {
28          duration: window.matchMedia("(prefers-reduced-motion: reduce)").matches ? 0 :
    1200,
29          step: function (value) { counter.text(Math.floor(value)); },
30          complete: function () { counter.text(target); }
31      });
32  });
```

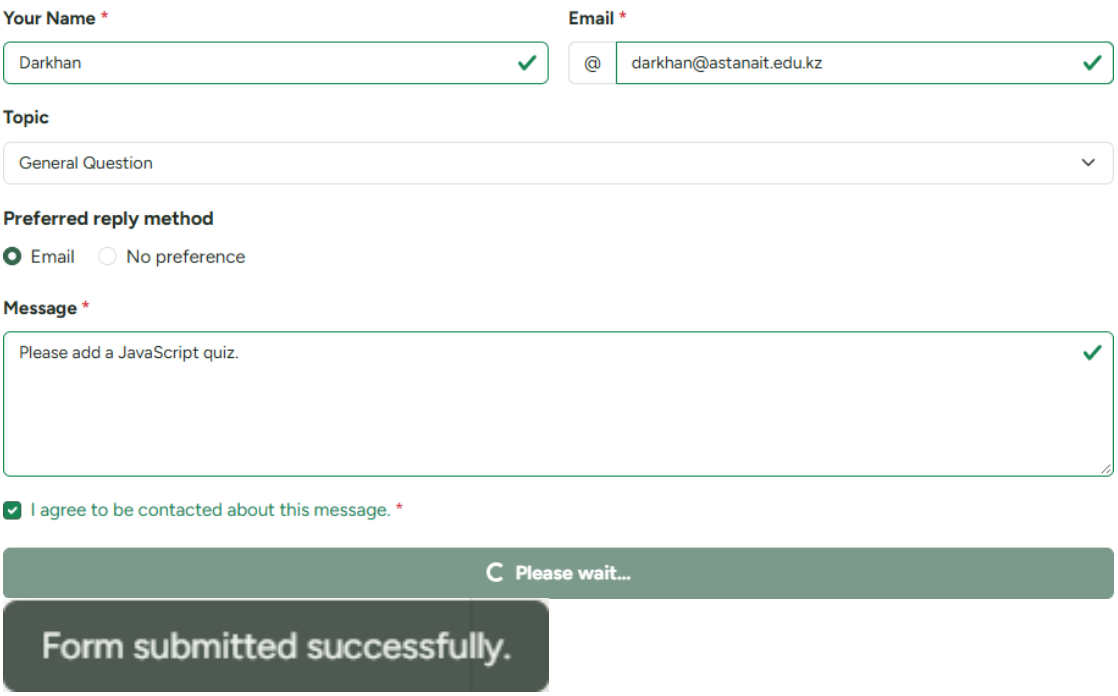
Tasks 6-7 / Submit spinner and notifications

1. Task 6: after validation succeeds in registration, newsletter or contact, disable its Submit button and show a spinner with Please wait... for 1.2 seconds.

2. Use a submitting flag to block repeated submissions. Then perform the original reset/success feedback and restore the button label and enabled state. Invalid forms show their usual field errors immediately.

3. Task 7: show a temporary success toast and fade it out automatically. Clear earlier animation queues when a new notification replaces the previous one.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment6.js : 39
39  function finishForm(form, boxId, message) {
40      var target = $(form);
41      if (target.data("submitting")) return;
42      var button = target.find('button[type="submit"]');
43      var label = button.html();
44      target.data("submitting", true).attr("aria-busy", "true");
45      button.prop("disabled", true).html('<span class="spinner-border spinner-border-sm
    me-2" aria-hidden="true"></span>Please wait…');
46      setTimeout(function () {
47          completeForm(form, boxId, message);
48          button.html(label).prop("disabled", false);
49          target.data("submitting", false).removeAttr("aria-busy");
50          showNotification("Form submitted successfully.");
51      }, 1200);
52  }
    js/assignment6.js : 54
54  function showNotification(message) {
55      $("#action-toast").stop(true, true).text(message).prop("hidden",
    false).hide().fadeIn(150).delay(2400).fadeOut(250);
56  }
```

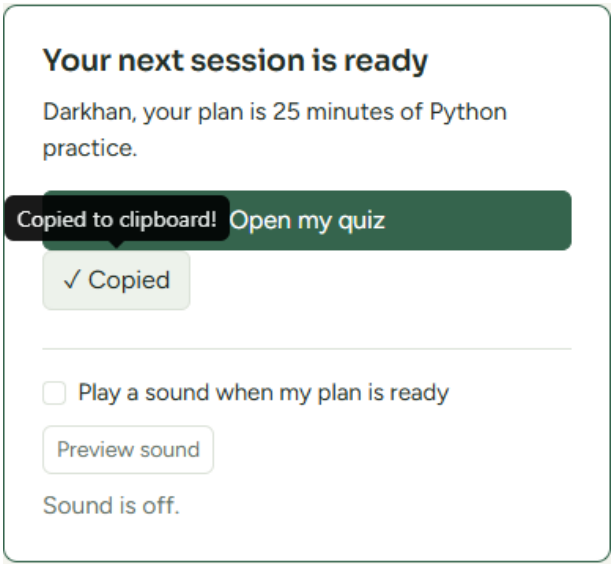
Task 8 / Copy plan to clipboard

1. Create a practice plan, then click Copy plan beside the generated summary. Copy the exact summary using the Clipboard API, with a textarea fallback for unsupported/denied access.

2. After a successful copy, change the button to a checkmark and Copied, show the Copied to clipboard! tooltip and a notification, then restore the label.

3. Do not claim success if both methods fail. Resetting the plan hides the Copy button and clears its feedback. Tests included real clipboard readback, fallback copying and the failure path.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment6.js : 97
97  function copied() {
98      resetCopyButton();
99      $(button).text("⎘ Copied");
100     var tooltip = bootstrap.Tooltip.getOrCreateInstance(button, { trigger: "manual",
    title: "Copied to clipboard!" });
101     tooltip.show();
102     showNotification("Copied to clipboard!");
103     $(button).data("copy-timer", setTimeout(resetCopyButton, 2000));
104 }
105 function fallback() {
106     var area = $("<textarea>").val(text).addClass("clipboard-buffer").appendTo("body");
    js/assignment6.js : 108
108     var success = false;
109     try { success = document.execCommand("copy"); } catch (error) { success = false; }
110     area.remove();
111     button.focus();
112     if (success) copied();
113     else showNotification("Copy failed. Select the plan text and copy it.");
114 }
    js/assignment6.js : 115
115 if (navigator.clipboard && window.isSecureContext)
    navigator.clipboard.writeText(text).then(copied, fallback);
116 else fallback();
```

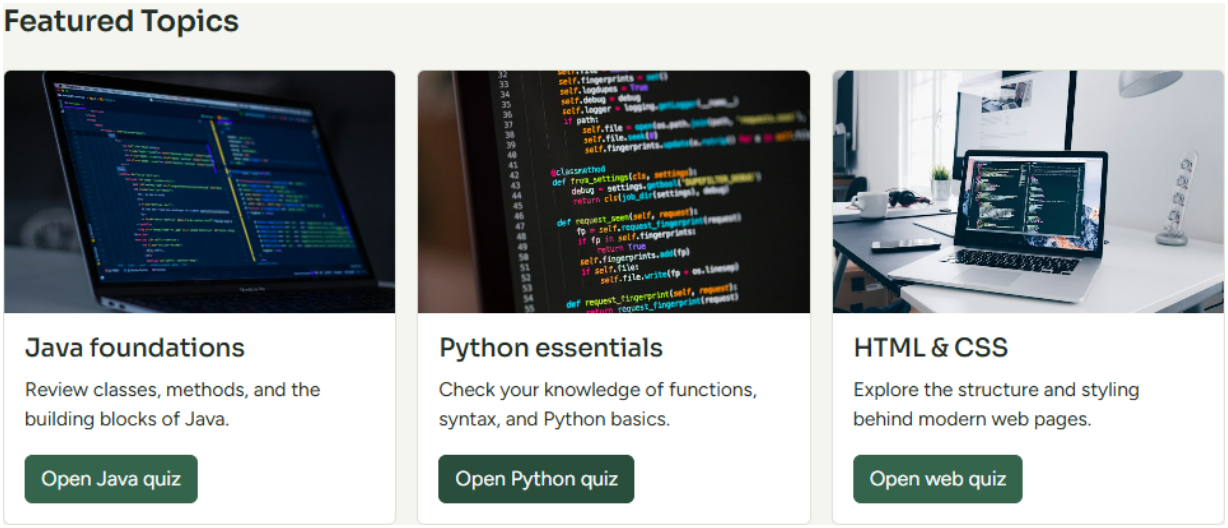
Task 9 / Image lazy loading

1. Keep each image URL in data-src and use a tiny placeholder initially. On scroll/resize, use jQuery visibility, offset and viewport boundaries to assign src only for images in view.

2. Recheck images when the carousel changes slides or a collapsed area opens. The nine carousel images load as their slides become visible.

3. When a gallery thumbnail changes the main image explicitly, remove the old lazy source so later scrolling cannot overwrite the selected picture. Tests recorded no offscreen request before scrolling and a real image request after exposure.

Result in the updated webpage



Source-code screenshot (selected lines)

```
    js/assignment6.js : 119
119 function initLazyImages() {
120     function loadVisible() {
121         var top = $(window).scrollTop();
122         var bottom = top + $(window).height();
123         $("img.lazy-image[data-src]").each(function () {
124             var image = $(this);
125             var imageTop = image.offset().top;
126             if (image.is(":visible") && imageTop < bottom && imageTop +
    image.outerHeight() > top) {
127                 image.attr("src", image.attr("data-src")).removeAttr("data-
    src").removeClass("lazy-image");
128             }
129         });
130     }
131     $(window).on("scroll resize load", loadVisible);
132     $(document).on("slid.bs.carousel shown.bs.collapse", loadVisible);
133     loadVisible();
134 }
```

Verification, reflection and deployment

Verification

51 test groups passed in Google Chrome 156.0.8078.74: 19 Assignment 6 groups, 25 prior-feature regression groups and 7 Forest platform groups. No JavaScript page errors were observed.

The suite checked live filtering, suggestions, repeated/unsafe queries, all search matches, scroll values, counter animation, reduced motion, loading/repeated submits, disappearing toasts, real clipboard readback, fallback/failure paths, lazy network requests, nine carousel slides and existing quizzes/planner/forms.

All five pages fit 320, 360, 390, 576, 768, 992 and 1440px viewports. Tests used localhost HTTP and offline file URLs with the CDN blocked. A physical phone and other browser engines are not claimed as tested.

Challenges and reflection

The main challenge was adding search without duplicating the existing topic data or breaking category filters. The existing render function now combines both criteria. Text-node highlighting protects existing markup and handles literal special characters. Reusing a single submission helper and notification function keeps the new behavior consistent. Lazy loading also needs explicit carousel and gallery handling so hidden images are delayed and selected images remain selected.

Part 4 / Deployment checklist

1. Copy the contents of CodeQuiz into the existing shared repository root, preserving its Git history.

2. Review the diff, commit and push all updated files. Rebuild GitHub Pages or redeploy to Netlify.

3. Open the public site and verify all five pages and Tasks 0-9 before submission.

4. Submit the PDF and complete project ZIP before the LMS deadline. Each member must submit and defend the work during practice.

Reproducible evidence

tests/assignment6-browser-tests.cjs; report/assignment-6/test-results.json; report/assignment-6/regression-results.json; report/assignment-6/screenshots/.